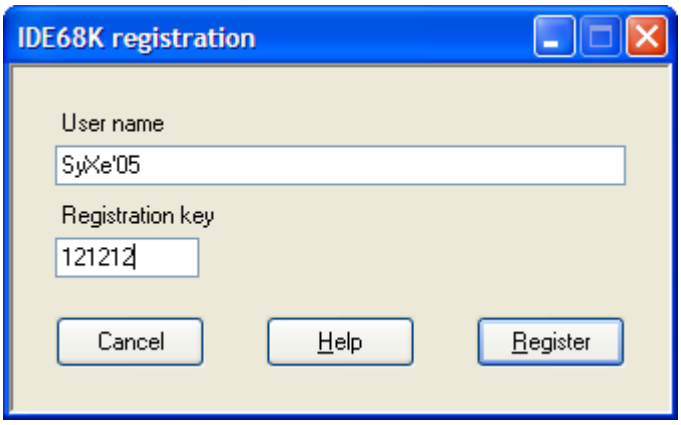
Programa:	68000 Integrated Development Environment 2.00 (Ide68k)
Protección:	Nagscreen
Objetivos:	Registrarnos con cualquier serial ;-)
URL descarga:	http://www.hzeeland.nl/~pfondse/ide68k/download.htm
Dificultad:	No es difícil
Herramientas:	OllyDbg v1.10 + Regmonitor + tiempo libre!
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 24-10-05

Hola a todos los CracksLatinoS..!!

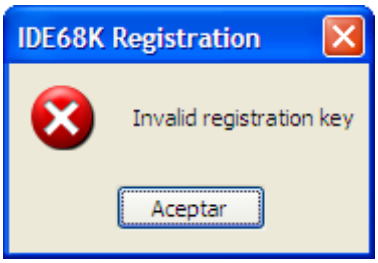
Enhorabuena por el trabajo que se está haciendo en la lista, se debe reconocer la mejora del grupo en cuanto a conocimientos se refiere, últimamente se están escribiendo muy buenos tutes, hoy quería presentar un programita del que me habló mi amigo Alfonso_iat_2004 el cual es un emulador de un micro motorola 68000, este va muy bien si eres estudiante de Informática (sistemas) y cursas 1° jeje.. te enseñan por encima como trabaja este Motorolla.

Bien, según Afonso (yo no lo he comprobado) la versión Trial (Shareware) te muestra una nagscreen de vez en cuando y cuando menos te lo esperas, esto lo hacía el ASDSee en versiones antiguas cuando superabas cierto límite de imágenes te sacaba una nag aleatoriamente, aquí por lo visto será similar, repito que yo no he podido ver la nag ¿? He ejecutado el programa y he estado un rato toqueteándolo todo oy no la he visto quizá haya que estar más tiempo pero bueno n o importa, el otro pero está en el About, veamos:

Vemos una referencia a evaluation y dice además Invalid Registration key ¿? No importa si miramos en el mismo directorio de instalación del programa encontramos otro ejecutable llamado Reg68k.exe que nos abre un cuadro de registro donde introducir un serial, jeje… esto se pone interesante pongamos uno:



Aceptamos pulsando el botón Register y nos muestra el típico mensaje de error:



Esto se pone todavía mejor, este mensaje nos va a ayudar bastante, aceptamos y se cierra todo no nos da más oportunidades, estudiemos un poco esto… abramos Olly y carguemos este ejecutable Reg68k.exe en él, damos RUN pero antes pondremos un BreakPoint en MessageBoxA por si utiliza esta API para mostrar el mensaje de error:

```
Command : bp MessageBoxA                    BP address, string -- Break with condition
```

Y al pulsar Enter el BP queda puesto, damos RUN y nos vuelve a aparecer la ventania de registro de antes, rellenamos los campos "User name" y "Registration key" aceptamos y Olly rompe, jeje… está llamando a la API como suponíamos, veamos el stack a ver:

```
0012FBBC  0040131B  CALL to MessageBoxA from Reg68k.00401316
0012FBC0  002C0668  hOwner = 002C0668 ('IDE68K registration',class='#32770')
0012FBC4  0040B31E  Text = "Invalid registration key"
0012FBC8  0040B2E7  Title = "IDE68K Registration"
0012FBCC  00000010  Style = MB_OK|MB_ICONHAND|MB_APPLMODAL
0012FBD0  0012FBEC
```

La llamada se hizo desde :401316 dentro del ejecutable Reg68k.exe, pulsamos Ctrl+F9 para retornar de la función, aparece el mensaje en pantalla, lo aceptamos y caemos ahora sí en el código del ejeutable a estudiar:

```
004011D7  .  6A 66          PUSH 66                          ControlID = 66 (102.)
004011D9  .  53             PUSH EBX                         hWnd
004011DA  .  E8 DF9D0000    CALL <JMP.&USER32.GetDlgItemTextA>  GetDlgItemTextA
004011DF  .  83F8 08        CMP EAX,8
004011E2  .v 74 20          JE SHORT Reg68k.00401204
004011E4  .  6A 10          PUSH 10                          Arg4 = 00000010
004011E6  .  FF35 ACB2400   PUSH DWORD PTR DS:[40B2AC]       Arg3 = 0040B2E7 ASCII "IDE68K Registration"
004011EC  .  FF35 B4B2400   PUSH DWORD PTR DS:[40B2B4]       Arg2 = 0040B31E ASCII "Invalid registration
004011F2  .  6A 00          PUSH 0                           Arg1 = 00000000
004011F4  .  E8 FD000000    CALL Reg68k.004012F6             Reg68k.004012F6
004011F9  .  83C4 10        ADD ESP,10
004011FC  .  6A 00          PUSH 0
```

La llamada se realizó en :4011F4 justo encima vemos una llamada a GetDlgItemTextA y detrás se verifica EAX si no tiene valor 8 no salta y llegamos al mensaje de error, esto quiere decir que debemos introducir un serial de 8 caracteres, repetimos la jugada pero ahora pondremos como serial 12121212 que cumple la restricción:

```
004011DA  .  E8 DF9D0000    CALL <JMP.&USER32.GetDlgItemTextA>  GetDlgItemTextA
004011DF  .  83F8 08        CMP EAX,8
004011E2  .v 74 20          JE SHORT Reg68k.00401204
004011E4  .  6A 10          PUSH 10                          Arg4 = 00000010
004011E6  .  FF35 ACB2400   PUSH DWORD PTR DS:[40B2AC]       Arg3 = 0040B2E7 ASCII "IDE68K Registration"
004011EC  .  FF35 B4B2400   PUSH DWORD PTR DS:[40B2B4]       Arg2 = 0040B31E ASCII "Invalid registration
004011F2  .  6A 00          PUSH 0                           Arg1 = 00000000
```

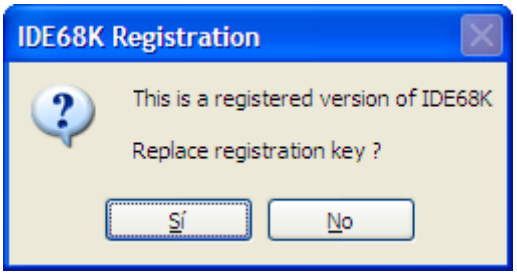
Evitamos la primera llamada al mensaje, si continuamos traceando llegamos a una llamada a EndDialog, después nos metemos en USER.dll, al final retornamos a:

```
0040135F  >  FF75 FC        PUSH DWORD PTR SS:[EBP-4]        hKey
00401362  .  E8 8D9C0000    CALL <JMP.&ADVAPI32.RegCloseKey>  RegCloseKey
00401367  .  6A 00          PUSH 0                           lParam = NULL
00401369  .  68 08114000    PUSH Reg68k.00401108             DlgProc = Reg68k.00401108
0040136E  .  6A 00          PUSH 0                           hOwner = NULL
00401370  .  68 29B44000    PUSH Reg68k.0040B429             pTemplate = "REG68K"
00401375  .  FF75 08        PUSH DWORD PTR SS:[EBP+8]        hInst
00401378  .  E8 6B9C0000    CALL <JMP.&USER32.DialogBoxParamA>  DialogBoxParamA
0040137D  .  85C0           TEST EAX,EAX
0040137F  .v 74 07          JE SHORT Reg68k.00401388
00401381  .  33C0           XOR EAX,EAX
00401383  .v E9 E0020000    JMP Reg68k.00401668
```

Salimos debajo de la llamada a DialogBoxParamA, continuamos traceando y llegamos a una parte donde antes no pasábamos:

```
004014D3  > F646 12 20      TEST BYTE PTR DS:[ESI+12],20
004014D7  .^ 0F84 76FFFFF   JE Reg68k.00401453
004014DD  >  F646 12 20     TEST BYTE PTR DS:[ESI+12],20
004014E1  .v 0F84 0C01000   JE Reg68k.004015F3
004014E7  .  6A 24          PUSH 24                          Arg4 = 00000024
004014E9  .  FF35 ACB2400   PUSH DWORD PTR DS:[40B2AC]       Arg3 = 0040B2E7 ASCII "IDE68K Registration"
004014EF  .  FF35 C4B2400   PUSH DWORD PTR DS:[40B2C4]       Arg2 = 0040B3A0 ASCII "This is a registered version
004014F5  .  56             PUSH ESI                         Arg1
```

hace un TEST con 20 de la variable [ESI+12] que está representando a [40C17A] y vale 47h que es una 'G' en la tabla ASCII, el salto se produce y evitamos la zona donde dice que es una versión registrada, no importa sigamos, vemos que es un bucle, al final deja de saltar y llegamos a la posterior comparación en :4014DD vemos que [ESI+12] vale 67h que es una 'g' el salto no se prroduce y pasamos por la zona del "Versión registrada…" esto parece ser un engaño o algo así porque no estamos registrados ni nada, si continuamos en el siguiente CALL nos muestra el siguiente mensaje:



nos dice pues eso que estamos ya registrados, he de decir que como ya tengo el parche quizá se haya escrito algo en el registro, puede ser pues no recuerdo que este mensaje apareciera antes, en cualquier caso como trabajamos con el .exe original sin cambios nos muestra el mensaje, sigamos lo aceptamos decimos que No y nos saca de la rutina con un salto casi al final de la misma y nos cierra el programa, hagamos una cosa el que tenga el programa recién instalado perfecto no debe de hacer nada y al que le pase lo que a mi cuando le aparezca el mensaje anterior que diga que sí, que quiere reemplazar el serial existente a ver si localizamos la rutina de verificación, llegamos a una zona donde encripta el serial con 0Ah:

```
004015F3  >   33DB          XOR EBX,EBX
004015F5  .v  EB 08         JMP SHORT Reg68k.004015FF
004015F7  >   80B3 DED6400  XOR BYTE PTR DS:[EBX+40D6DE],0A
004015FE  .   43            INC EBX
004015FF  >   68 DED64000   PUSH Reg68k.0040D6DE          ASCII "12121212"
00401604  .   E8 27020000   CALL Reg68k.00401830
00401609  .   59            POP ECX
0040160A  .   3BC3          CMP EAX,EBX
0040160C  .^  77 E9         JA SHORT Reg68k.004015F7
0040160E  .   33DB          XOR EBX,EBX
00401610  .v  EB 08         JMP SHORT Reg68k.0040161A
```

Ese XOR lo encripta, si hacemos Follow in dump vemos nuestro serial normalmente:

```
Address   Hex dump                                         ASCII
0040D6DE  31 32 31 32 31 32 31 32 00 00 00 00 00 00 00 00  12121212........
0040D6EE  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040D6FE  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040D70E  00 00 00 00 00 00 00 00 00 00 A4 1C 40 00 A4 1C  ..........ŕL@.ŕL
```

Allí lo vemos normal sin encriptar, cuando finalice el loop miramos el resultado:

```
Address   Hex dump                                         ASCII
0040D6DE  3B 38 3B 38 3B 38 3B 38 00 00 00 00 00 00 00 00  ;8;8;8;8........
0040D6EE  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040D6FE  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040D70E  00 00 00 00 00 00 00 00 00 00 A4 1C 40 00 A4 1C  ..........ŕL@.ŕL
```

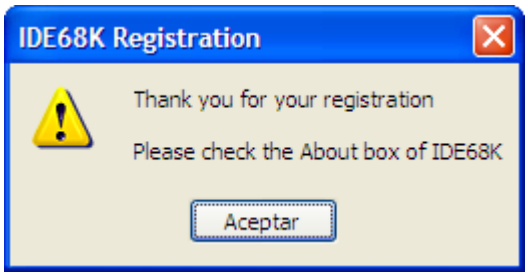
nuestro 121212 se transformó en esta cadena ;8;8;8;8 que es el propio serial pero encriptado, sigamos más adelante a ver qué más hace, de momento vemos que también encripta el nombre con 05h:

```
00401612  >   80B3 ACD6400  XOR BYTE PTR DS:[EBX+40D6AC],5
00401619  .   43            INC EBX
0040161A  >   68 ACD64000   PUSH Reg68k.0040D6AC          ASCII "SyXe'05"
0040161F  .   E8 0C020000   CALL Reg68k.00401830
00401624  .   59            POP ECX
00401625  .   3BC3          CMP EAX,EBX
00401627  .^  77 E9         JA SHORT Reg68k.00401612
```

Al final queda esto:

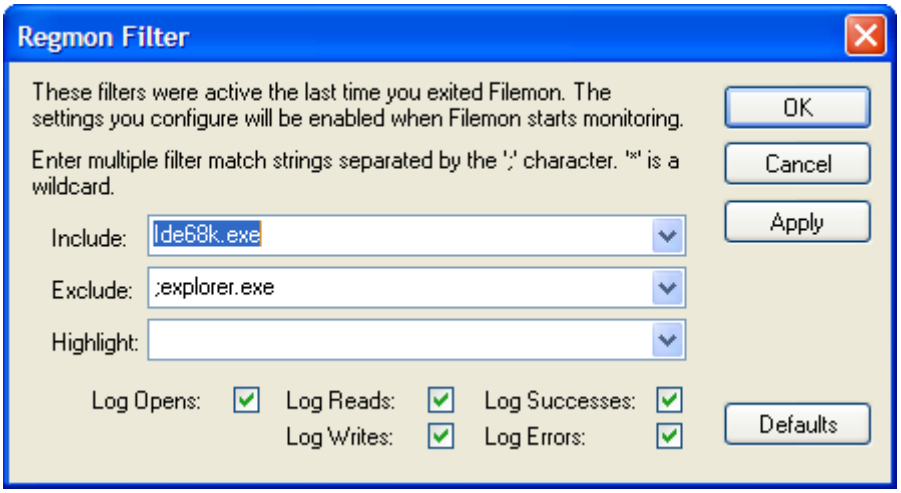
```
Address   Hex dump                                         ASCII
0040D6AC  56 7C 5D 60 22 35 30 00 00 00 00 00 00 00 00 00  V|]`"50.........
0040D6BC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040D6CC  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
0040D6DC  00 00 3B 38 3B 38 3B 38 3B 38 00 00 00 00 00 00  ..;8;8;8;8......
```

Eso quedó de SyXe'05, jeje… se está entreteniendo en encriptarlo todo, que querrá hacer lo vamos a descubrir pronto, después de esto finaliza su trabajo se cierra y nos da este mensaje:



O sea que para saber si es o no válido el serial debemos mirar en la ventanita de About, pero ojo que ahora estamos trabajando con el ejecutable "Reg68k.exe" y debemos mirarlo en "Ide68k.exe" lo que hacemos es cerrar este Olly y abrir otro con Ide68k.exe, o bien sin cerrar Olly pulsar F3 y lo cargamos, desde aquí vamos a buscar el momento en que valida el serial, sabemos que el ejecutable "Reg68k.exe" se encarga simplemente de encriptar los datos y almacenarlos en algún lugar para que al ejecutar Reg68k.exe lo recoja, lo desencripte y lo valide, esto es así, entonces a partir de ahora como los datos ya están almacenados (seguramente en el registro o en algún archivo .ini) trabajaremos con Ide68k.exe y trataremos de localizar el momento exacto en el que desencripta y valida estos datos de registro, si conseguimos parar la ejecución en ese instante las posibilidades de éxito aumentan considerablemente.

No perdamos más tiempre y carguemos el .exe en Olly, para averiguar donde guarda el nombre y el serial nos vamos a valer de la estupenda herramienta Regmonitor, lo configuramos de esta manera:



le estamos diciendo que sólo loguee los movimientos de Ide68k.exe e ignore los del resto de archivos, así nos saldrá un listado con sólo lo esencial y del programa que queremos, aceptamos y se abre el Regmon quedando a la espera de eventos ;-) es el momento de ejecutar Ide68k.exe y ver que obtenemos recompensa:

#	Time	Process	Request	Path	Result	Other
832	5.92523419	Ide68...	DeleteValu...	HKCU\Software\IDE68K\2.0\Window8	NOTFOUND	
833	5.92524849	Ide68...	DeleteValu...	HKCU\Software\IDE68K\2.0\Window9	NOTFOUND	
834	5.92536221	Ide68...	SetValue	HKCU\Software\IDE68K\2.0\Window0	SUCCESS	"C:\Ide68k\Examples\Newf
835	5.96010881	Ide68...	CloseKey	HKCU\Software\IDE68K\2.0	SUCCESS	Key: 0xE11B3908
836	6.38991545	Ide68...	OpenKey	HKLM\Software\IDE68K\2.0	SUCCESS	Key: 0xE14E4300
837	6.38994417	Ide68...	SetValue	HKLM\Software\IDE68K\2.0\Commport	SUCCESS	"COM1=9600,8,0,0,0"
838	6.39032623	Ide68...	CloseKey	HKLM\Software\IDE68K\2.0	SUCCESS	Key: 0xE14E4300
839	6.39067075	Ide68...	CreateKey	HKCU\Software\IDE68K\2.0	SUCCESS	Key: 0xE14E4300
840	6.39216341	Ide68...	SetValue	HKCU\Software\IDE68K\2.0\Window	SUCCESS	"66"
841	6.39262821	Ide68...	SetValue	HKCU\Software\IDE68K\2.0\(Default)	SUCCESS	"SyXe'05"
842	6.39264743	Ide68...	SetValue	HKCU\Software\IDE68K\2.0\Project	SUCCESS	""
843	6.39267294	Ide68...	DeleteValu...	HKCU\Software\IDE68K\2.0\File0	NOTFOUND	

Vemos que es una llamada a SetValue pero no importa esto nos confirma nuestras sospechas de que guarda el nombre y serial en el registro, eso sí poniendo BP's en las apis RegQueryValueA y RegQueryValueExA no obtenemos nada ¿? Es evidente que utiliza alguna otra fórmula para recuperar los datos de registro, tampoco nos sirven los BP's en GetPrivateProfileIntA ni GetPrivateProfileStringA, o se que no parece que lo lea de ningún archivo con extensión .ini de todas maneras tenemos que pillarle como sea, no puede ser que nos tire abajo tan fácil, seguiremos luchando hasta el fin ;-)

Lo que hice fue una búsqueda de las cadenas (strings) del ejecutable (recordad que estamos trabajando con una copia virgen de Ide68k.exe) y mirad que encontré:

```
Text strings referenced in Ide68k:CODE
Address   Disassembly                                  Text string
0040592A  PUSH Ide68k.0042D32D                         ASCII "%d"
00405953  PUSH Ide68k.0042D5B2                         ASCII "%02d"
0040597C  PUSH Ide68k.0042D32D                         ASCII "%d"
004059B4  PUSH Ide68k.0042D58F                         ASCII "Edit"
004059E9  PUSH Ide68k.0042D5B7                         ASCII "68000 IDE - %s"
00405A10  PUSH Ide68k.0042D2D0                         ASCII "68000 IDE"
00405A36  PUSH Ide68k.0042D5EA                         ASCII "Unauthorized copy of IDE68K"
00405A3B  PUSH Ide68k.0042D5C6                         ASCII "Run SETUP to re-install the program"
00405DCE  MOV ESI,Ide68k.0042D765                      ASCII "!?@#$%&*()/+-={}[]ab"
00405DE6  PUSH Ide68k.0042D70C                         ASCII ";8;8;8;8V|]`"50"
00405E12  PUSH Ide68k.0042D70C                         ASCII ";8;8;8;8V|]`"50"
00405E29  MOV EDI,Ide68k.0042D714                      ASCII "V|]`"50"
0040608D  PUSH Ide68k.0042D60C                         ASCII "https://www.yaskifo.com/register.asp?LNG=US&ID=102259
00406092  PUSH Ide68k.0042DB45                         ASCII "open"
004060A7  PUSH Ide68k.0042DB4A                         ASCII "Cannot connect to the Internet"
004060E3  PUSH Ide68k.0042DB69                         ASCII "RegisterDlg"
00406149  MOV DWORD PTR DS:[43624C],Ide68k.0042DB      ASCII "Open file"
0040615D  MOV DWORD PTR DS:[436258],Ide68k.0042DB      ASCII "asm"
0040617F  PUSH Ide68k.0042DB86                         ASCII "rb"
```

¿no os suenan estas cadenas? Si hombre recordad:

serial encriptado: ;8;8;8;8
nombre encriptado: V|]'"50

Si juntamos ambas cadenas que obtenemos? Jeje.. pues justo esta ;8;8;8;8V|]`"50 que es la que encontramos entre las strings que guarda, como aparece más de una vez ponemos un BP en cada referencia, cuando lo tengamos listo damos RUN y para en la primera de ellas:

```
00405DDF  .  6A 14          PUSH 14                          Arg3 = 00000014
00405DE1  .  8D4424 14      LEA EAX,DWORD PTR SS:[ESP+14]
00405DE5  .  50             PUSH EAX                         Arg2
00405DE6  .  68 0CD74200    PUSH Ide68k.0042D70C             Arg1 = 0042D70C ASCII ";8;8;8;8V|]`"50"
00405DEB  .  E8 A0C10100    CALL Ide68k.00421F90             Ide68k.00421F90
00405DF0  .  83C4 0C        ADD ESP,0C
00405DF3  .  85C0           TEST EAX,EAX
00405DF5  .v 75 19          JNZ SHORT Ide68k.00405E10
00405DF7  .  C705 705C430   MOV DWORD PTR DS:[435C70],1AE5D8
00405E01  .  C705 745C430   MOV DWORD PTR DS:[435C74],4BDDA
00405E0B  .v E9 87010000    JMP Ide68k.00405F97
00405E10  >  6A 08          PUSH 8                           Arg3 = 00000008
00405E12  .  68 0CD74200    PUSH Ide68k.0042D70C             Arg2 = 0042D70C ASCII ";8;8;8;8V|]`"50"
00405E17  .  8D5424 0C      LEA EDX,DWORD PTR SS:[ESP+C]
00405E1B  .  52             PUSH EDX                         Arg1
00405E1C  .  E8 57C20100    CALL Ide68k.00422078             Ide68k.00422078
00405E21  .  83C4 0C        ADD ESP,0C
00405E24  .  33C0           XOR EAX,EAX
00405E26  .  83C9 FF        OR ECX,FFFFFFFF
00405E29  .  BF 14D74200    MOV EDI,Ide68k.0042D714          ASCII "V|]`"50"
00405E2E  .  C64424 0C 00   MOV BYTE PTR SS:[ESP+C],0
```

Vemos además que están las dos muy seguidas, son los dos puntos en los que aparece la cadena (serial+nombre encriptados) cuando para ya ha empujado a la pila algunos valores:

```
0012FEE8  0012FF00  ASCII "!?@#$%&*()/+-={}[]ab"
0012FEEC  00000014
0012FEF0  00000000
0012FEF4  0015C658
0012FEF8  02080026
```

Ya ha metido la cadena que vemos y un "14" luego hace un PUSH de nuestros datos de registro encriptados cadena_reg a partir de ahora para no liarnos mucho, de momento entramos en el CALL a ver qué es lo que hace con estos valores:

```
00421F99  .  8BF7          MOV ESI,EDI
00421F9B  .  8B45 10       MOV EAX,DWORD PTR SS:[EBP+10]
00421F9E  .  8BC8          MOV ECX,EAX
00421FA0  .v E3 1E         JECXZ SHORT Ide68k.00421FC0
00421FA2  .  8BD0          MOV EDX,EAX
00421FA4  .  66:33C0       XOR AX,AX
00421FA7  .  F2:AE         REPNE SCAS BYTE PTR ES:[EDI]
00421FA9  .  2BD1          SUB EDX,ECX
00421FAB  .  8BCA          MOV ECX,EDX
00421FAD  .  8BFE          MOV EDI,ESI
00421FAF  .  8B75 08       MOV ESI,DWORD PTR SS:[EBP+8]
00421FB2  .  F3:A6         REPE CMPS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
00421FB4  .  33C0          XOR EAX,EAX
00421FB6  .  8BD0          MOV EDX,EAX
00421FB8  .  8A46 FF       MOV AL,BYTE PTR DS:[ESI-1]
00421FBB  .  8A57 FF       MOV DL,BYTE PTR DS:[EDI-1]
00421FBE  .  2BC2          SUB EAX,EDX
00421FC0  >  5F            POP EDI
00421FC1  .  5E            POP ESI
00421FC2  .  5D            POP EBP
00421FC3  .  C3            RETN
```

Atención que esta comparando, esta instrucción "REPE…" va comparando carácter a carácter las cadenas que hay en EDI y en ESI (tanto EDI como ESI contienen un puntero a una cadena) estas las compara de esta manera, lo más interesante de todo está lógicamente en los registros, veamos que tienen ESI y EDI:

```
Registers (FPU)
EAX 00000000
ECX 00000014
EDX 00000014
EBX 00151F04
ESP 0012FED4
EBP 0012FEDC
ESI 0042D70C ASCII ";8;8;8;8V!]'"50"
EDI 0012FF00 ASCII "!?@#$%&*()/+-={}[]ab"
EIP 00421FB2 Ide68k.00421FB2
```

Está más que claro verdad? ESI contiene un puntero a la cadena_red y EDI a otra cadena más que sospechosa, de esto se deduce que el nombre más el serial deben sumar un total de caracteres igual al de la segunda cadena !?@#$%&*()/+-={}[]ab que tiene 20, o sea que el nombre + el serial deben sumar 20 caracteres o al menos esto parece ser, luego se deben encriptar:

Serial con 0Ah y nombre con 05h, después el resultado de cada uno se concatena en una cadena única que se compara con la anterior, estamos suponiendo por su puesto que la cadena anterior es válida, pero quizá tras la comparación aunque sean iguales no nos acepte el serial, por eso como es una suposición seguiremos traceando… vemos que deshace los pasos anteriores que hizo al encriptar los datos cuando los introducimos en el cuadro de registro ya que aquí desencripta el serial:

```
00405E54  .  33C0          XOR EAX,EAX
00405E56  .  890424        MOV DWORD PTR SS:[ESP],EAX
00405E59  .  895424 28     MOV DWORD PTR SS:[ESP+28],EDX
00405E5D  .v EB 0E         JMP SHORT Ide68k.00405E6D
00405E5F  >  8B4C24 28     MOV ECX,DWORD PTR SS:[ESP+28]
00405E63  .  8031 0A       XOR BYTE PTR DS:[ECX],0A          ;desencripta serial...
00405E66  .  FF0424        INC DWORD PTR SS:[ESP]
00405E69  .  FF4424 28     INC DWORD PTR SS:[ESP+28]
00405E6D  >  8D4424 04     LEA EAX,DWORD PTR SS:[ESP+4]
00405E71  .  50            PUSH EAX
00405E72  .  E8 4DC00100   CALL Ide68k.00421EC4
00405E77  .  59            POP ECX
00405E78  .  3B0424        CMP EAX,DWORD PTR SS:[ESP]
00405E7B  .^ 77 E2         JA SHORT Ide68k.00405E5F
```

al final queda el serial desencriptado en la ventana de datos (ventana de dump) y un poco después hace lo propio con el nombre que pusimos:

```
00405E8C  >  8B4C24 2C     MOV ECX,DWORD PTR SS:[ESP+2C]
00405E90  .  8031 05       XOR BYTE PTR DS:[ECX],5           ;desencripta nombre...
00405E93  .  FF0424        INC DWORD PTR SS:[ESP]
00405E96  .  FF4424 2C     INC DWORD PTR SS:[ESP+2C]
00405E9A  >  68 345A4300   PUSH Ide68k.00435A34              ASCII "V!]'"50"
00405E9F  .  E8 20C00100   CALL Ide68k.00421EC4
00405EA4  .  59            POP ECX
00405EA5  .  3B0424        CMP EAX,DWORD PTR SS:[ESP]
00405EA8  .^ 77 E2         JA SHORT Ide68k.00405E8C
```

Al pasar por estos dos bucles quedan los datos desencriptados nuevamente, luego se mete en un bucle donde va tratando todos y cada uno de los caracteres del nombre, les aplica algunas operaciones y va almacenando el resultado en la variable [435C70], es aquí:

```
00405EC3  >  68 345A4300   PUSH Ide68k.00435A34              ASCII "SyXe'05"
00405EC8  .  E8 F7BF0100   CALL Ide68k.00421EC4
00405ECD  .  59            POP ECX
00405ECE  .  8BD8          MOV EBX,EAX
00405ED0  .  2B1C24        SUB EBX,DWORD PTR SS:[ESP]
00405ED3  .  8B4424 30     MOV EAX,DWORD PTR SS:[ESP+30]
00405ED7  .  0FBE10        MOVSX EDX,BYTE PTR DS:[EAX]
00405EDA  .  52            PUSH EDX                          Arg1
00405EDB  .  E8 20200200   CALL Ide68k.00427F00              Ide68k.00427F00
00405EE0  .  59            POP ECX
00405EE1  .  0FAFD8        IMUL EBX,EAX
00405EE4  .  69CB 15CD5B07 IMUL ECX,EBX,75BCD15
00405EEA  .  010D 705C430( ADD DWORD PTR DS:[435C70],ECX     ;va almacenando...
00405EF0  .  FF0424        INC DWORD PTR SS:[ESP]
00405EF3  .  FF4424 30     INC DWORD PTR SS:[ESP+30]
00405EF7  >  68 345A4300   PUSH Ide68k.00435A34              ASCII "SyXe'05"
00405EFC  .  E8 C3BF0100   CALL Ide68k.00421EC4
00405F01  .  59            POP ECX
00405F02  .  3B0424        CMP EAX,DWORD PTR SS:[ESP]
00405F05  .^ 77 BC         JA SHORT Ide68k.00405EC3
```

Va almacenando en ese "ADD DWORD PTR [435C70], ECX" donde ECX tiene el resultado de las operaciones de cada carácter, cuando se han procesado todos queda una cadena:

```
Address   Hex dump                                          ASCII
00435C70  05 E9 FA 46 00 00 00 00 00 00 00 00 00 00 00 00  ♣ú·F............
00435C80  00 00 00 00 00 00 00 00 40 06 60 00 98 06 21 00  ........@♠`.¯♠!.
00435C90  A2 06 28 00 B2 07 2F 00 00 00 00 00 00 00 00 00  ó♠(.▒·/.........
00435CA0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 40 00  ..............@.
```

Se generó la cadena "46FAE905" lógicamente el que ponga otro nombre recogerá otra distinta pero siempre estará ahí en [435C70] ya que es una variable global, si salimos del bucle donde acabamos de estar y traceamos nada un pelín nos metemos en otro:

```
00405F1E  > 8B4424 34    MOV EAX,DWORD PTR SS:[ESP+34]
00405F22  . 0FBE10       MOVSX EDX,BYTE PTR DS:[EAX]
00405F25  . F60455 BE494 TEST BYTE PTR DS:[EDX*2+4349BE],4
00405F2D  .v 74 0C       JE SHORT Ide68k.00405F3B
00405F2F  . 8B4C24 34    MOV ECX,DWORD PTR SS:[ESP+34]
00405F33  . 0FBE01       MOVSX EAX,BYTE PTR DS:[ECX]
00405F36  . 83C0 D0      ADD EAX,-30
00405F39  .v EB 11       JMP SHORT Ide68k.00405F4C
00405F3B  > 8B5424 34    MOV EDX,DWORD PTR SS:[ESP+34]
00405F3F  . 0FBE0A       MOVSX ECX,BYTE PTR DS:[EDX]
00405F42  . 51           PUSH ECX                                 Arg1
00405F43  . E8 B81F0200  CALL Ide68k.00427F00                     Ide68k.00427F00
00405F48  . 59           POP ECX
00405F49  . 83C0 C9      ADD EAX,-37
00405F4C  > 8B15 745C430 MOV EDX,DWORD PTR DS:[435C74]
00405F52  . C1E2 04      SHL EDX,4
00405F55  . 03C2         ADD EAX,EDX
00405F57  . A3 745C4300  MOV DWORD PTR DS:[435C74],EAX            ;va almacenando...
00405F5C  . FF0424       INC DWORD PTR SS:[ESP]
00405F5F  . FF4424 34    INC DWORD PTR SS:[ESP+34]
00405F63  > 8D4424 04    LEA EAX,DWORD PTR SS:[ESP+4]
00405F67  . 50           PUSH EAX
00405F68  . E8 57BF0100  CALL Ide68k.00421EC4
00405F6D  . 59           POP ECX
00405F6E  . 3B0424       CMP EAX,DWORD PTR SS:[ESP]
00405F71  .^ 77 AB       JA SHORT Ide68k.00405F1E
```

Es muy similar al anterior sólo que ahora trata el serial y no el nombre como hizo antes, es de resaltar que el funcionamiento es el mismo tras las operaciones va almacenando el resultado en [435C74] que es un DWORD más que el anterior, esto quiere decir que irán pegadas en la ventana de datos cuando lo miremos, a la salida del bucle vemos la cadena que se generó para ese serial 12121212:

```
Address   Hex dump                                          ASCII
00435C54  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00435C64  00 00 00 00 00 00 00 00 00 00 00 00 05 E9 FA 46  .............‡ú·F
00435C74  12 12 12 12 00 00 00 00 00 00 00 00 00 00 00 00  ‡‡‡‡............
00435C84  00 00 00 00 40 06 60 00 98 06 21 00 A2 06 28 00  ....@♠`.˜♠!.¢♠(.
00435C94  B2 07 2F 00 00 00 00 00 00 00 00 00 00 00 00 00  ▒•/.............
00435CA4  00 00 00 00 00 00 00 00 00 00 40 00 A0 05 0A 5E  ..........@.à♣.^
00435CB4  F6 11 0A 52 FA 0F 0A 36 AE 11 0A 58 84 08 0A 4D  ÷◄.R·♫.6«◄.X■♦.M
00435CC4  57 10 0A 94 28 00 8A 01 00 00 00 00 00 00 00 00  W►.ö(.è0........
```

jeje… no se generó ningún serial sólo lo pasó a un registro, claro por eso pide un seiral de 8 caracteres porque es el máximo que puede albergar un registro de 32 bits, pues resumiendo lo que hicieron los loops fue el primero generar una cadena a partir del nombre y el segundo simplemente pasar el serial a la variable [435C74] en formato hexadecimal claro está, seguimos traceando y se mete en un CALL donde hay otro bucle, lo más interesante es que retorna un valor en EAX que es guardado en la misma variable donde estaba el serial [435C74] chafándolo después del CALL, es aquí:

```
00405F79  .  68 B14FFFFF  PUSH FFFF4FB1                          Arg3 = FFFF4FB1
00405F7E  .  81E9 BA9E000 SUB ECX,9EBA
00405F84  .  68 3B620000  PUSH 623B                              Arg2 = 0000623B
00405F89  .  51           PUSH ECX                               Arg1
00405F8A  .  E8 E5FDFFFF  CALL Ide68k.00405D74                   Ide68k.00405D74
00405F8F  .  83C4 0C      ADD ESP,0C
00405F92  .  A3 745C4300  MOV DWORD PTR DS:[435C74],EAX
00405F97  >  83C4 38      ADD ESP,38
00405F9A  .  5F           POP EDI
00405F9B  .  5E           POP ESI
00405F9C  .  5B           POP EBX
00405F9D  .  C3           RETN
```

Vemos el "MOV DWORD PTR [435C74], EAX" que chafa nuestro serial y coloca el valor que tiene EAX, en este caso vale 076C095E, este valor quedará almacenado en [435C74] después viene un RET que nos pone en la zona decisiva del programa:

```
0040144E  >  E8 75490000  CALL Ide68k.00405DC8
00401453  .  A1 705C4300  MOV EAX,DWORD PTR DS:[435C70]
00401458  .  3B05 745C430 CMP EAX,DWORD PTR DS:[435C74]
0040145E  .v 75 08        JNZ SHORT Ide68k.00401468
00401460  .  33D2         XOR EDX,EDX
00401462  .  8915 24D2420 MOV DWORD PTR DS:[42D224],EDX
00401468  >  833D A003450 CMP DWORD PTR DS:[4503A0],0
0040146F  .v 74 08        JE SHORT Ide68k.00401479
```

Bouuuuuuuuufff pero que vemos!! Recupera el valor de la primera variable [435C70] que almacena el valor que se generó a partir del nombre SyXe'05 y que era 46FAE905, deja en EAX este valor y luego la instrucción más importante de todas ¡La comparación! Jeje… compara el contenido de ambas variables a través del registro EAX, tenemos que:

[435C70]==46FAE905
[435C74]==57CD06A5

Bien podríamos estudiar el algoritmo de generación de ambas para sacarle el KeyGen (yo aún no he hecho ninguno) pero esto quedará para otro momento, ahora nos podemos centrar en forzar al programa a aceptar cualquier serial que introduzcamos como si fuera válido ;-)

Se supone que el valro de ambas variables debe ser el mismo, está claro que si ambos coinciden entonces es serial válido (lo he podido comprobar en el About, jeje…) ahora hay que hacer que ambos coincidan, aprovecharemos las líneas de código entre :401453 y :40145E (ambas incluídas) allí mismo podemos modificar un poco el código para dejar las cosas en su sitio ;-D

El parche que podemos hacer será algo así:

```
0040144E  > E8 75490000  CALL Ide68k.00405DC8
00401453  . A1 705C4300  MOV EAX,DWORD PTR DS:[435C70]
00401458    A3 745C4300  MOV DWORD PTR DS:[435C74],EAX
0040145D    90           NOP
0040145E  .v 75 08       JNZ SHORT Ide68k.00401468
00401460  . 33D2         XOR EDX,EDX
```

Salvamos las modificaciones como Ide68k_crk.exe y lo ejecutamos, como la nag aparece cuando le viene en gana a ella miraremos en el About a ver si hay cambios ;-)

jejeje… vaya si hay cambios, aceptó el serial como válido sin serlo realmente, esto imdica que el parche fincionó e hizo su trabajo, lo único que hicimos fue meter en una variable lo que hay en la otra nada más ;-DD

Con este parche tan sencillo vencemos la protección, como no trae más limitaciones ni Time-Trial ni nada podemos decir que hemos terminado el tabajito de hot, para cualquier cosa ya sabéis donde encontrarme:

syxe00@yahoo.es

un slaudo para todo CracksLatinoS sin excepción y uno en particular para Alfonso_iat que fue quién me habló de este programa, un saludo amigo y otro para todos vosotros los CracksLatinoS, hasta la próxima, chao…!!!

SyXe'05